

BCSE202L- Data Structures and Algorithm

Module 2

Dr. G. Praveen Kumar
Assistant Professor, Gr2
SMEC, SCOPE

Module:1	Algorithm Analysis	8 hours
Importance of algorithms and data structures - Fundamentals of algorithm analysis: Space and time complexity of an algorithm, Types of asymptotic notations and orders of growth - Algorithm efficiency – best case, worst case, average case - Analysis of non-recursive and recursive algorithms - Asymptotic analysis for recurrence relation: Iteration Method, Substitution Method, Master Method and Recursive Tree Method.		
Module:2	Linear Data Structures	7 hours
Arrays: 1D and 2D array- Stack - Applications of stack: Expression Evaluation, Conversion of Infix to postfix and prefix expression, Tower of Hanoi – Queue - Types of Queue: Circular Queue, Double Ended Queue (deQueue) - Applications – List: Singly linked lists, Doubly linked lists, Circular linked lists- Applications: Polynomial Manipulation.		
Module:3	Searching and Sorting	7 hours
Searching: Linear Search and binary search – Applications. Sorting: Insertion sort, Selection sort, Bubble sort, Counting sort, Quick sort, Merge sort - Analysis of sorting algorithms.		
Module:4	Trees	6 hours
Introduction - Binary Tree: Definition and Properties - Tree Traversals- Expression Trees:- Binary Search Trees - Operations in BST: insertion, deletion, finding min and max, finding the k^{th} minimum element.		

Module:5 Graphs	6 hours
Terminology – Representation of Graph – Graph Traversal: Breadth First Search (BFS), Depth First Search (DFS) - Minimum Spanning Tree: Prim's, Kruskal's - Single Source Shortest Path: Dijkstra's Algorithm.	
Module:6 Hashing	4 hours
Hash functions - Separate chaining - Open hashing: Linear probing, Quadratic probing, Double hashing - Closed hashing - Random probing – Rehashing - Extendible hashing.	
Module:7 Heaps and AVL Trees	5 hours
Heaps - Heap sort- Applications -Priority Queue using Heaps. AVL trees: Terminology, basic operations (rotation, insertion and deletion).	
Module:8 Contemporary Issues	2 hours

Text Book

1. Mark A. Weiss, Data Structures & Algorithm Analysis in C++, 4th Edition, 2013, Pearson Education.

Reference Books

1. Alfred V. Aho, Jeffrey D. Ullman and John E. Hopcroft, Data Structures and Algorithms, 1983, Pearson Education.
2. Horowitz, Sahni and S. Anderson-Freed, Fundamentals of Data Structures in C, 2008, 2nd Edition, Universities Press.
3. Thomas H. Cormen, C.E. Leiserson, R L. Rivest and C. Stein, Introduction to Algorithms, 2009, 3rd Edition, MIT Press.

Context :

Importance of algorithms and data structures - Fundamentals of algorithm analysis: Space and time complexity of an algorithm, Types of asymptotic notations and orders of growth

- RAM (Random Access Memory) is the **volatile memory** of a computer, which stores **data and instructions** temporarily while programs are running.
- Data in RAM is stored in **contiguous memory locations**.
- Each location in RAM has a **unique address**.
- Arrays are stored in these memory locations, element by element.



Arrays- 1D & 2D

An **array** is a collection of elements stored in **contiguous memory locations**. Each element in the array is accessed using an **index**, which starts from **0**.

Array Operation

Operation

Traversal

Insertion

Deletion

Searching

Sorting

Updating

Copying

Description

Visit each element

Add an element

Remove an element

Find an element

Order elements

Modify an element

Duplicate array

1. Traversal

```
for (int i = 0; i < size; i++) {  
    printf("%d ", arr[i]);  
}
```

2. Insertion

```
int pos = 2, val = 99;  
for (int i = size; i > pos; i--) {  
    arr[i] = arr[i - 1];  
}  
arr[pos] = val;  
size++;
```

3. Deletion

```
int pos = 2;
for (int i = pos; i < size - 1; i++) {
    arr[i] = arr[i + 1];
}
size--;
```

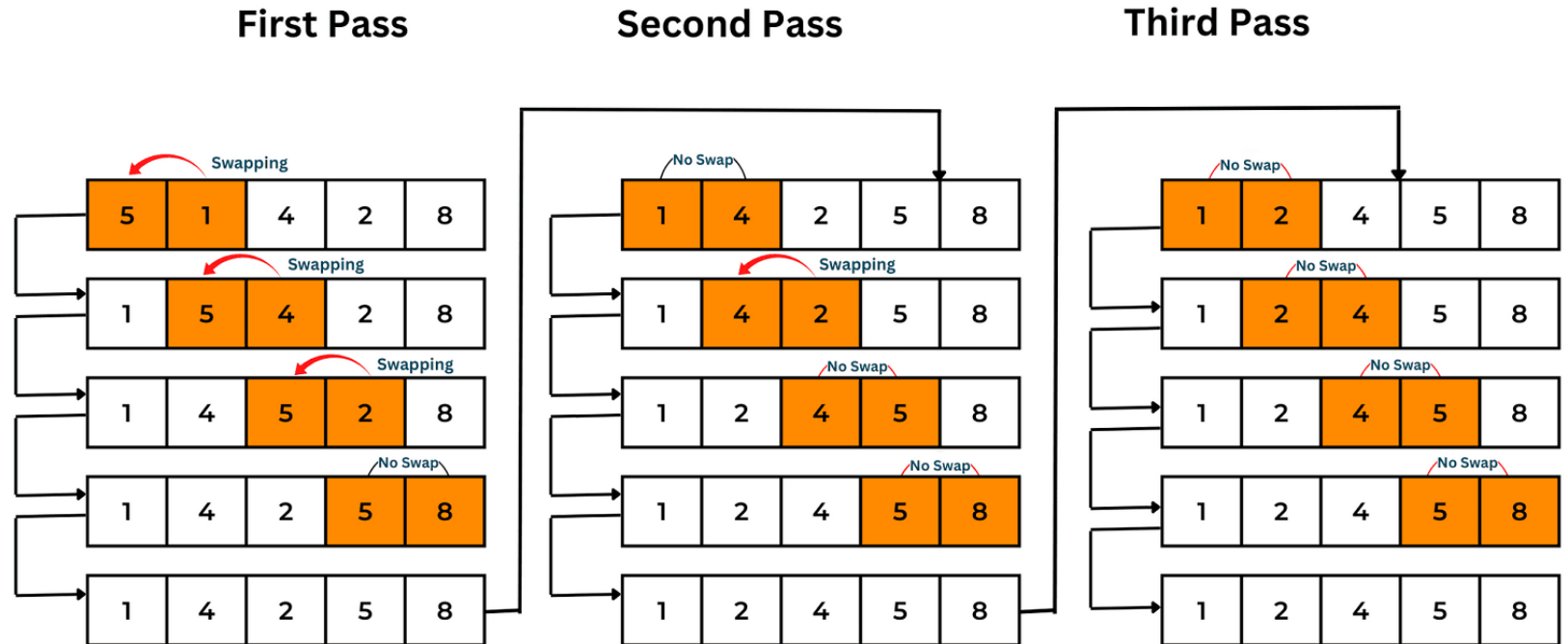
4. Searching

```
int key = 30;
for (int i = 0; i < size; i++) {
    if (arr[i] == key) {
        printf("Found at index %d", i);
    }
}
```


5. Sorting

```
for (int i = 0; i < size - 1; i++) {  
    for (int j = 0; j < size - i - 1; j++) {  
        if (arr[j] > arr[j + 1]) {  
            int temp = arr[j];  
            arr[j] = arr[j + 1];  
            arr[j + 1] = temp;  
        }  
    }  
}
```

BUBBLE SORTING



Arrays- 1D & 2D

```
#include <stdio.h>
#include <string.h>

int main() {
    char str[100];
    int i, count = 0;

    printf("Enter a string: ");
    fgets(str, sizeof(str), stdin); // read input with spaces

    for (i = 0; str[i] != '\0'; i++) {
        char ch = str[i];
        // Check for vowels (both lowercase and uppercase)
        if (ch == 'a' || ch == 'e' || ch == 'i' || ch == 'o' || ch == 'u' ||
            ch == 'A' || ch == 'E' || ch == 'I' || ch == 'O' || ch == 'U') {
            count++;
        }
    }

    printf("Number of vowels in the string: %d\n", count);
    return 0;
}
```

Arrays- 1D & 2D

A one-dimensional (1D) array is a linear collection of elements, like a single row or list.

A two-dimensional (2D) array, is a grid-like structure, like a table, with rows and columns

1D Array in RAM

A 1D array (one-dimensional array) is a list of elements of the same data type, stored in contiguous memory locations.

```
int arr[5] = {10, 20, 30, 40, 50};
```

Assume the base address of arr is 1000, and int takes 4 bytes.

Element	Value	Address
arr[0]	10	1000
arr[1]	20	1004
arr[2]	30	1008
arr[3]	40	1012
arr[4]	50	1016

```
printf("%d", arr[2]); // prints 30
```

2D Array in RAM

A 2D array is an array of arrays, like a matrix with rows and columns.

```
int matrix[2][3] = {  
    {1, 2, 3},  
    {4, 5, 6}  
};
```

- C stores 2D arrays in **row-major order**: row by row.
- Assume base address is **2000**, and int is 4 bytes.

Element	Value	Address
matrix[0][0]	1	2000
matrix[0][1]	2	2004
matrix[0][2]	3	2008
matrix[1][0]	4	2012
matrix[1][1]	5	2016
matrix[1][2]	6	2020

```
printf("%d", matrix[1][2]); // prints 6
```

2D Array in RAM

Declaration and Initialization

```
//Array declaration
char ch[10];
int x[10];
float f[10];
//Initialization
x[10] = {1,2,3,4,5};
ch[10] = {'M', 'G', 'C', 'U'};
f[10] = {3.2, 5.3, 1.2};

//Declaration and Initialization
int x[10] = {1,2,3,4,5};
char ch[10] = {'M', 'G', 'C', 'U'};
float f[10] = {3.2, 5.3, 1.2};

//Dynamic Initialization (without size)
int x[] = {1,2,3,4,5};
char ch[] = {'M', 'G', 'C', 'U'};
float f[] = {3.2, 5.3, 1.2};
```

Graphical Representation

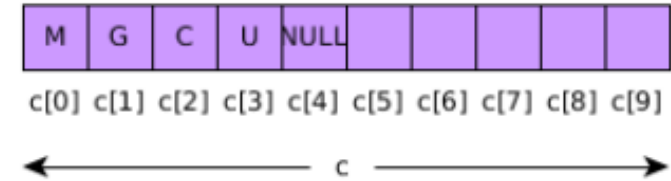


Figure 2

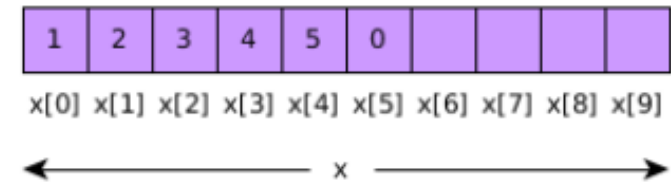
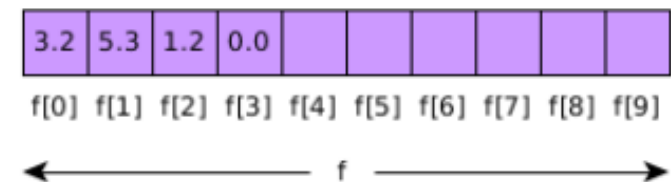


Figure 3



2D Array in RAM

Two Dimensional Array

- Two dimensional array is stored in row-column format
- First indices indicate row and second indices indicate column
- Similarly, multi-dimensional arrays are possible

Declaration and Initialization

```
//Declaration and Initialization
int x[2][3] = {{1, 3, 0}, {-1, 5, 9}};
int x[][3] = {{1, 3, 0}, {-1, 5, 9}};
int x[2][3] = {1, 3, 0, -1, 5, 9};
```

```
//Initialization with missing values
int y[5][3] = {1, 2, 3, 4, 5, 6, 7};
```

1	3	0
-1	5	9

←x[2][3]→

Graphical Representation

y[0][0]	y[0][1]	y[0][2]
y[1][0]	y[1][1]	y[1][2]
y[2][0]	y[2][1]	y[2][2]
y[3][0]	y[3][1]	y[3][2]
y[4][0]	y[4][1]	y[4][2]

←y[5][3]→

Figure 6

1	2	3
4	5	6
7	0	0
0	0	0
0	0	0

←y[5][3]→

Properties of Matrices:

- **Commutative Law of Addition:** $A + B = B + A$.
- **Associative Law of Addition:** $(A + B) + C = A + (B + C)$.
- **Associative Law of Multiplication:** $(AB)C = A(BC)$.
- **Distributive Law:** $A(B + C) = AB + AC$ and $(A + B)C = AC + BC$.
- **Identity Matrix:** $AI = A = IA$.
- **Transpose Properties:** $(A^T)^T = A$

Properties of Matrices: Matrix Addition (2D Array)

```
#include <stdio.h>

int main() {
    int rows, cols;

    // Ask user for matrix size
    printf("Enter number of rows and columns: ");
    scanf("%d %d", &rows, &cols);

    // Input elements for Matrix 1
    printf("Enter elements of Matrix 1:\n");
    for (int i = 0; i < rows; ++i) {
        for (int j = 0; j < cols; ++j) {
            printf("Enter element at [%d][%d]: ", i, j);
            scanf("%d", &matrix1[i][j]);
        }
    }

    // Adding the two matrices
    for (int i = 0; i < rows; ++i) {
        for (int j = 0; j < cols; ++j) {
            sum[i][j] = matrix1[i][j] + matrix2[i][j];
        }
    }

    // Display the result
    printf("Sum of the two matrices:\n");
    for (int i = 0; i < rows; ++i) {
        for (int j = 0; j < cols; ++j) {
            printf("%d\t", sum[i][j]);
        }
        printf("\n");
    }

    return 0;
}
```


Properties of Matrices: Transpose Matrix (2D Array)

```
#include <stdio.h> /* Transpose Matrix*/
int main()
{
    int r, c, a[50][50], i, j;
    printf("Enter number of rows and column: \t");
    scanf("%d%d", &r, &c);
    printf("Enter elements of 1st matrix:\n");
    for (i = 0; i < r; ++i)
        for (j = 0; j < c; ++j)
            scanf("%d", &a[i][j]);

    printf("\nTranspose matrix: \n");
    for (i = 0; i < c; ++i)
    {
        for (j = 0; j < r; ++j)
            printf("%d\t", a[j][i]);
        printf("\n");
    }
    return 0;
}
```

Structure:

- Unlike arrays, A structure is a user-defined data type that allows grouping variables of different data types under a single name
- Data elements in a structure are called fields or members
- Structures are also called records
- A structure type in C is called struct.
- Structures are useful for representing complex entities or records where multiple related pieces of information, potentially of varying types, need to be grouped together.

○ Student information:

- student id,
- last name,
- first name
- major,
- gender,

○ Bank account information:

- account number,
- account type
- account holder
 - first name
 - last name
- balance

```
struct Student {  
    int id;  
    char name[50];  
    float marks;  
};
```

Structure:

How to declare variable of type structure?

Create a variable, var1, using this structure:

```
typedef struct{  
    type1 id1;  
    type2 id2;  
    ...  
} struct_name ;
```

```
struct_name var1;
```

Examples:

```
person p1;  
p1.name;  
p1.age;
```

Accessing Members

Use array indexing + dot operator:

Structure:

```
#include <stdio.h>
```

```
// Define structure with typedef
```

```
typedef struct {
```

```
    int day;
```

```
    int month;
```

```
    int year;
```

```
} eventDate;
```

```
int main() {
```

```
    // Declare structure variable
```

```
    eventDate ev1;
```

```
// Get input from user
```

```
    printf("Enter day: ");
```

```
    scanf("%d", &ev1.day);
```

```
    printf("Enter month: ");
```

```
    scanf("%d", &ev1.month);
```

```
    printf("Enter year: ");
```

```
    scanf("%d", &ev1.year);
```

```
    // Display values
```

```
    printf("Event Date: %02d-%02d-%04d\n", ev1.day, ev1.month,  
ev1.year);
```

```
    return 0;
```

```
}
```

Array of Structures:

An **array of structures** is a collection of multiple structure variables stored in contiguous memory locations, where each element of the array is a structure of the same type.

This allows to manage and manipulate groups of related data efficiently (e.g., student records, inventory items, employee details).

- Structure student S[10] ;

	Name	Class	Roll_number	Marks [5]
S[0]	S[0].name	S[0].class	S[0].roll_number	S[0].marks[5]
S[1]	S[1].name	S[1].class	S[1].roll_number	S[1].marks[5]
⋮	⋮	⋮	⋮	⋮
S[9]	S[9].name	S[9].class	S[9].roll_number	S[9].marks[5]

An Array of Structure

Hierarchical Structure

A hierarchical structure in C refers to nested or embedded structures, where one structure contains another structure as a member.

```
#include <stdio.h>
#include <string.h>

// Sub-structure
typedef struct {
    int day;
    int month;
    int year;
} Date;

// Main structure (hierarchical)
typedef struct {
    char name[30];
    int rollNo;
    Date dob; // Nested structure
} Student;
```

```
Student s1
├── name      → "Rahul"
├── rollNo    → 101
└── dob
    ├── day   → 15
    ├── month → 8
    └── year   → 2000
```

Types of C Functions

- **Library function** Are the in-built function in C programming system
- **User defined function** Allows programmer to define their own function according to their requirement.

Some of the math.h Library Functions

- `sin()` → returns the sine of a radian angle.
- `cos()` → returns the cosine of an angle in radians.
- `tan()` → returns the tangent of a radian angle.
- `floor()` → returns the largest integral value less than or equal to x.

Some of the conio.h Library Functions

- `clrscr()` → used to clear the output screen.
- `getch()` → reads character from keyboard.

Types of C Functions

Advantages of user defined functions

- It helps to decompose the large program into small segments which makes programmer easy to understand, maintain and debug.
- If repeated code occurs in a program. Function can be used to include those codes and execute when needed by calling that function.
- Programmer working on large project can divide the workload by making different functions

Function naming rule in C

- Name of function includes only alphabets, digit and underscore.
- First character of name of any function must be an alphabet or underscore.
- Name of function cannot be any keyword of c program.
- Name of function cannot be global identifier.
- Name of function cannot be exactly same as of name of function in the same scope.
- Name of function is case sensitive
- Name of function cannot be register Pseudo variable

Types of C Functions

`calculate_sum` - Alphabets/underscore only

`_print_data` - Starts with underscore

`processData2` - Contains digit (not first character)

`2nd_function` - Starts with digit

`float()` - Uses C keyword

`printf()` - Conflicts with global identifier

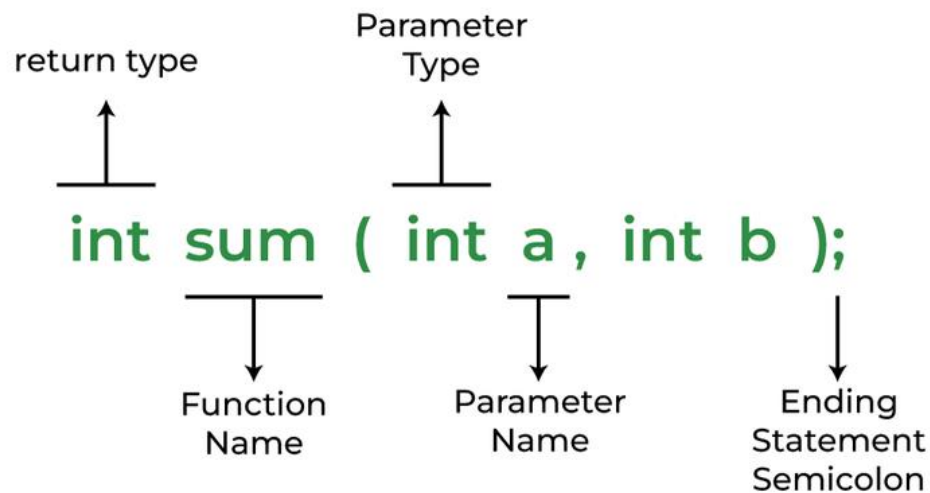
Duplicate `calculate_sum()` in same scope

`Calculate_Sum()` - Allowed (different case from `calculate_sum`)

`register()` - Uses reserved storage specifier

4 ways to declare functions

Function Type	Description
No arguments, no return value	Performs a task without input data and doesn't return a result
Arguments, no return value	Accepts input parameters but doesn't return a value
No arguments, with return value	Returns a value without requiring input parameters
Arguments and return value	Accepts input parameters and returns a processed result



4 ways to declare functions

```
#include <stdio.h>
```

```
// Function definition
```

```
void printMessage() {  
    printf("Hello!\n");  
}
```

```
int main() {  
    printMessage(); // Function call  
    return 0;  
}  
#include <stdio.h>  
#include <stdlib.h> // For rand()
```

```
// Function definition
```

```
int getRandom() {  
    return rand() % 100; // Random number between 0 and 99  
}
```

```
int main() {  
    int num = getRandom(); // Function call and storing return value  
    printf("Random number: %d\n", num);  
    return 0;  
}
```

```
#include <stdio.h>
```

```
// Function definition
```

```
void greetUser(char name[]) {  
    printf("Hello, %s!\n", name);  
}
```

```
int main() {  
    greetUser("Alice"); // Function call with argument  
    return 0;  
}
```

```
#include <stdio.h>
```

```
// Function definition
```

```
int add(int a, int b) {  
    return a + b;  
}
```

```
int main() {  
    int result = add(10, 20); // Function call with arguments  
    printf("Sum = %d\n", result);  
    return 0;  
}
```

Stack

A stack is a linear data structure that follows the LIFO (Last In, First Out) principle.

A LIFO (last-in first-out) storage structure. The first thing you put in is the last thing you take out. The last thing you put in is the first thing you take out.

Operations:

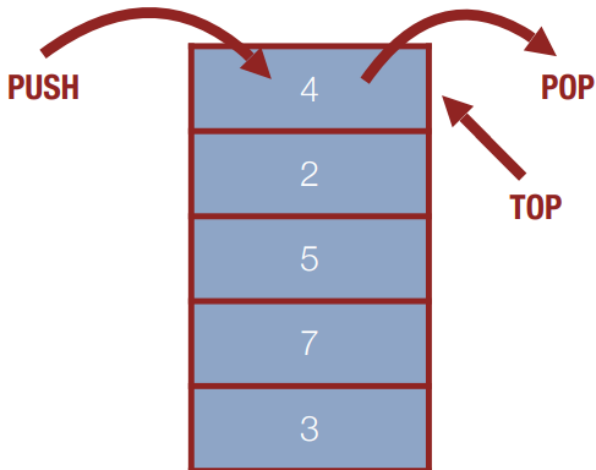
push() – adds an element to the top

pop() – removes the top element

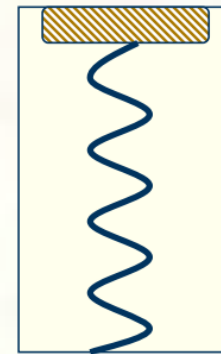
peek() or top() – returns the top element without removing it

isEmpty() – checks if the stack is empty

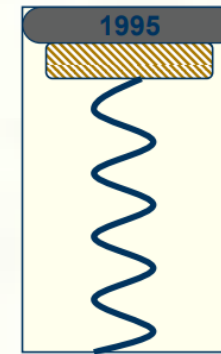
isFull() – (in case of fixed-size array) checks if the stack is full



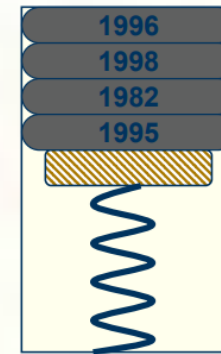
■ Coin rest in the arm of an automobile



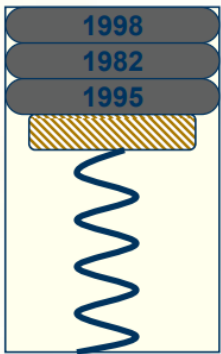
Initial State



After
One Push

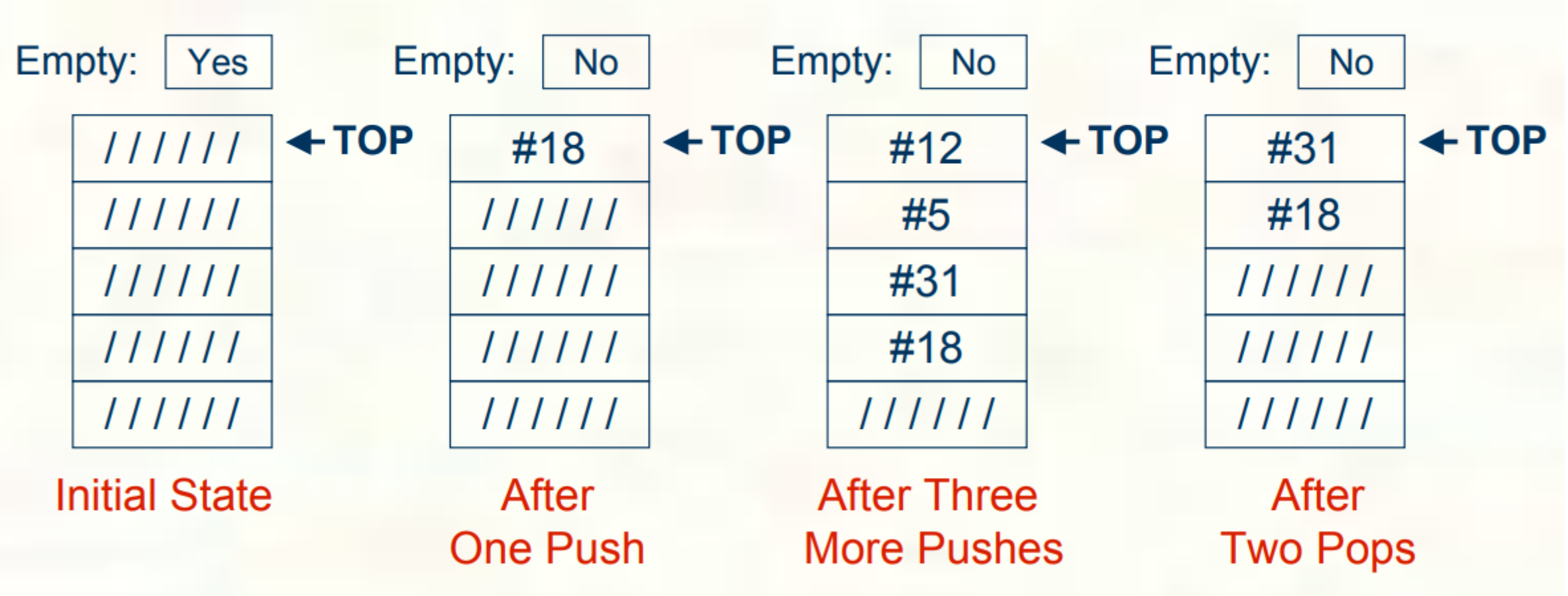


After Three
More Pushes



After
One Pop

Stack



Stack

Limitation	Explanation
✗ No Random Access	You can't directly access an element in the middle or at the bottom. Only the top is accessible.
✗ No Traversal	Unlike arrays or linked lists, you can't loop through all elements unless you pop them one by one.
✗ No Search	Searching through a stack is not practical. You'd have to remove elements until you find the one you're looking for.

LIFO is Natural for Many Problems: Many real-world and programming problems like undo-redo, parsing, and recursion are naturally LIFO.

APPLICATIONS OF STACKS

- 1- Check for balancing of parenthesis.
- 2- Convert infix expression to postfix.
- 3- Evaluate postfix expression.
- 4- Use of Stack in Function calls.
- 5- Check for palindrome strings (E.g: "madam""racecar""level""noon""121")

PARENTHESIS

A stack is useful here because we know that when a closing symbol such as **)** is seen, it matches the most recently seen unclosed **(**, therefore, by placing an opening symbol on a stack, we can easily determine whether a closing symbol makes sense. Specifically, we have the following algorithm.

1-Make an **empty** stack.

2-Read symbols until the **end** of the file.

a-If the symbol is an **opening symbol**, push it onto the stack. **(, [, {**

b-If it is a **closing symbol**, **do** the following: **),], }**

i-If the stack is **empty**, **report** an error

ii-Otherwise, **pop** the stack

- if the symbol popped is **not** the corresponding opening symbol, **report** an error.

-else **continue**.

3- After processing the entire string: If the stack is empty, the expression is balanced. If not, it is unbalanced.

Stack

PARENTHESIS

{x + (y - [a + b]) * c}

Step	Character	Stack Operation	Stack Content
1	{	Push {	{
2-3	x, +	Ignore (not brackets)	{
4	(	Push (	{ (
5-6	y, -	Ignore	{ (
7	[	Push [	{ ([
8-10	a + b	Ignore	{ ([
11	]	Match with [→ Pop	{ (
12	)	Match with (→ Pop	{
13-14	* c	Ignore	{
15	}	Match with { → Pop	(empty)

Stack

PARENTHESIS

```
#include <stdio.h>
#include <stdlib.h>
```

```
#define SIZE 100
```

```
char stack[SIZE];
int top = -1;

void push(char ch) {
    if (top < SIZE - 1)
        stack[++top] = ch;
}
```

```
char pop() {
    if (top >= 0)
        return stack[top--];
    return '\0';
}
```

```
int isMatchingPair(char open, char close) {
    return (open == '(' && close == ')') ||
           (open == '{' && close == '}') ||
           (open == '[' && close == ']');
}

int isBalanced(char *expr) {
    for (int i = 0; expr[i] != '\0'; i++) {
        char ch = expr[i];
        if (ch == '(' || ch == '{' || ch == '[') {
            push(ch);
        } else if (ch == ')' || ch == '}' || ch == ']') {
            if (top == -1 || !isMatchingPair(pop(), ch))
                return 0; // Not balanced
        }
    }
    return (top == -1); // If stack is empty, it's balanced
}
```

```
int main() {
    char expression[SIZE];
    printf("Enter expression: ");
    scanf("%s", expression);

    if (isBalanced(expression))
        printf("Balanced\n");
    else
        printf("Not Balanced\n");

    return 0;
}
```

Stack

What is the result?

$$5 * 4 - 8 / 2 + 9$$

25

Algebraic Expression

- An algebraic expression is a legal combination of operands and the operators.
- Operand is the quantity (unit of data) on which a mathematical operation is performed.
- Operand may be a variable like x, y, z or a constant like 5, 4, 0, 9, 1 etc.
- Operator is a symbol which signifies a mathematical or logical operation between the operands. Example of familiar operators include +, -, *, /, ^

Stack

INFIX: $x+y$, $6*3$

- **POSTFIX:** Postfix notation are also Known as Reverse Polish Notation (RPN).

They are different from the infix and prefix notations in the sense that in the postfix notation, operator comes after the operands, e.g. $xy+$, $xyz+^*$ etc.

- **PREFIX:** Prefix notation also Known as Polish notation.

In the prefix notation, as the name only suggests, operator comes before the operands, e.g. $+xy$, $+xyz$ etc.

Operator Priorities

Priority Level	Operators (Precedence rule)	Associativity Rule
Highest (1)	() Parenthesis	Right to left
High (1)	$\wedge$	Right to left
2	$*, /$	Left to Right
3	$+, -$	Left to Right
Lowest	$=$	Right to Left

Stack

Infix	
A+B	
$(A+B) * (C + D)$	
$A-B/(C*D^E)$	

Stack **Algorithm for Expression, Infix to Postfix**

- 1) Initialize (Create an empty stack for operators, Create an empty result string for postfix expression, Define operator precedence rules)
- 2 Examine the next element in the input.
- 3) If it is operand, output it.
- 3a) If it is opening parenthesis, push it on stack.
- 4) If it is an operator, then
 - i) If stack is empty, push operator on stack.
 - ii) If the top of stack is opening parenthesis, push operator on stack
 - iii) If it has higher priority than the top of stack, push operator on stack.
 - iv) Else pop the operator from the stack and output it, repeat step 4.
- 5) If it is a closing parenthesis, pop operators from stack and output them until an opening parenthesis is encountered. pop and discard the opening parenthesis.
- 6) If there is more input go to step 1
- 7) If there is no more input, pop the remaining operators to output.

Stack Algorithm for Expression, Infix to Postfix

1. $K + L - M * N + (O \wedge P) * W / U / V * T + Q$

Stack

Algorithm for Infix to Postfix

convert $2*3/(2-1)+5*3$ into Postfix form,

No.	Infix notation
1	$A + B$
2	$(A + B) * C$
3	$A * (B + C)$
4	$A / B + C / D$
5	$(A + B) * (C + D)$
6	$((A + B) * C) - D$

Prefix notation	Postfix notation
$+ A B$	$A B +$
$*+ A B C$	$A B + C *$
$*A + B C$	$A B C + *$
$+ / A B / C D$	$A B / C D / +$
$*+ A B + C D$	$A B + C D + *$
$-*+ A B C D$	$A B + C * D -$

Stack

Use of Stack in Function calls

- Manages nested function calls.
- Helps handle recursion.
- Maintains execution order and local variables.

```
void funcA() {  
    funcB();  
}
```

```
void funcB() {  
    // do something  
}
```

```
int factorial(int n) {  
    if (n == 0) return 1;  
    return n * factorial(n - 1);  
}
```

```
factorial(3)  
factorial(2)  
factorial(1)  
factorial(0)
```

```
void func1() {  
    int a = 10;  
    func2();  
}
```

```
void func2() {  
    int b = 20;  
    // use b  
}
```


Stack

C program for Recursion function

```
#include <stdio.h>
```

```
unsigned long long factorial(int n) {  
    // Base cases  
    if (n == 0 || n == 1) {  
        return 1;  
    }  
    // Recursive case  
    return n * factorial(n - 1);  
}
```

```
int main() {  
    int num;  
    printf("Enter a positive integer to calculate its factorial: ");  
    scanf("%d", &num);  
  
    // Input validation  
    if (num < 0) {  
        printf("Factorial is not defined for negative numbers.\n");  
    } else {  
        unsigned long long result = factorial(num);  
        printf("Factorial of %d = %llu\n", num, result);  
    }  
  
    return 0;  
}
```

Uses unsigned long long to handle larger factorial values (up to 20! safely, as 21! exceeds 64-bit limits).

Stack

Tower of Hanoi

The **Tower of Hanoi** is a classic mathematical puzzle involving three rods and a number of disks of different sizes, which can slide onto any rod. The puzzle starts with the disks stacked in ascending order of size on one rod (smallest at the top, largest at the bottom) and the goal is to move the entire stack to another rod, following these rules:

- **Only one disk can be moved at a time.**
- **A disk can only be placed on top of a larger disk or on an empty rod.**
- **All disks must maintain their original order** (smallest on top, largest on bottom) on the destination rod at the end.

Let's consider a Tower of Hanoi with 3 disks on rod A, to be moved to rod C, using rod B as auxiliary. The disks are labeled 1 (smallest), 2, and 3 (largest).

Stack

Tower of Hanoi

Initial State:

Rod A: [1, 2, 3] (smallest on top)

Rod B: []

Rod C: []

Steps (following the recursive algorithm):

Move disk 1 from A to C.

A: [2, 3], B: [], C: [1]

Move disk 2 from A to B.

A: [3], B: [2], C: [1]

Move disk 1 from C to B.

A: [3], B: [1, 2], C: []

Move disk 3 from A to C.

A: [], B: [1, 2], C: [3]

Move disk 1 from B to A.

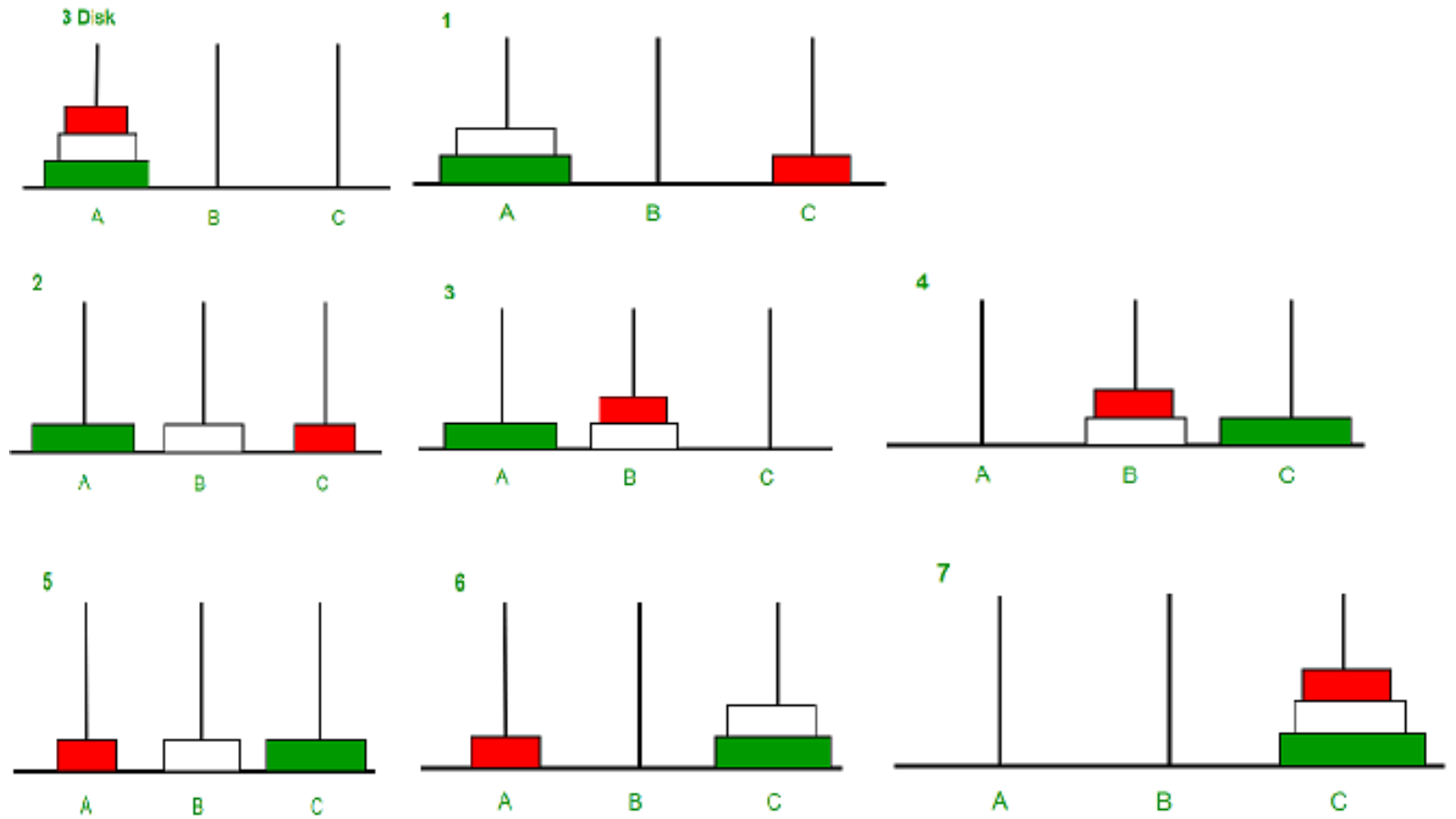
A: [1], B: [2], C: [3]

Move disk 2 from B to C.

A: [1], B: [], C: [2, 3]

Move disk 1 from A to C.

A: [], B: [], C: [1, 2, 3]



Minimum Moves:

For n disks, the minimum number of moves is $2^n - 1$.

For example: 1 disk: 1 move, 2 disks: 3 moves, 3 disks: 7 moves, 4 disks: 15 moves, and so on.

```
FUNCTION Tower Of Hanoi (n, source, auxiliary, destination)
```

```
IF n == 1 THEN
```

```
    PRINT "Move disk 1 from " + source + " to " + destination
```

```
    RETURN
```

```
END IF
```

```
// Move (n-1) disks from source to auxiliary
```

```
CALL TowerOfHanoi(n - 1, source, destination, auxiliary)
```

```
// Move the nth disk from source to destination
```

```
PRINT "Move disk " + n + " from " + source + " to " + destination
```

```
// Move the (n-1) disks from auxiliary to destination
```

```
CALL TowerOfHanoi(n - 1, auxiliary, source, destination)
```

```
END FUNCTION
```

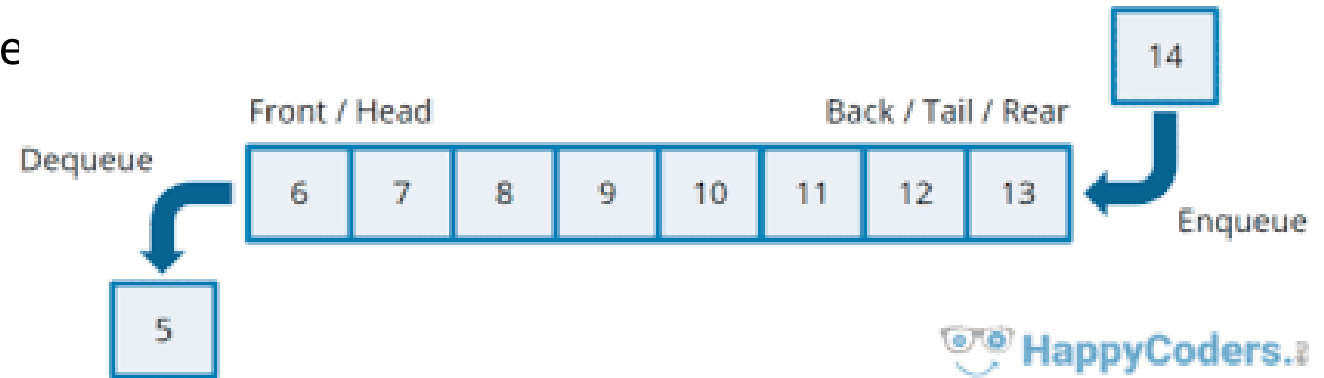
Queue

A queue is a linear data structure that follows the First-In-First-Out (FIFO) or LIFO principle, where the first element added is the first one to be removed.

Implementation of Queue using array

A Queue is implemented using a static array with two pointers: front (points to the first element) and rear (points to the last element). Elements are enqueued at the rear and dequeued from the front.

- **Basic Operations: Enqueue:** Adding an element to the rear of the queue.
- **Dequeue:** Removing an element from the front of the
- **Front or Peek:** Accessing the element at the front.
- **Rear:** Accessing the element at the rear.
- **isEmpty/isFull:** Checking the queue's status.



 HappyCoders.in

e.g., boarding a bus, printer jobs

Challenges: Once rear reaches the array's end, no more elements can be added, even if space is available at the beginning after dequeuing (wasted space).

Implementation of Queue using array

```
#include <stdio.h>
#define SIZE 5

int queue[SIZE];
int front = -1, rear = -1;

// Enqueue operation
void enqueue(int value) {
    if (rear == SIZE - 1) {
        printf("Queue Overflow!\n");
    } else {
        if (front == -1 && rear == -1)
            front = rear = 0;
        queue[rear] = value;
        printf("Inserted %d\n", value);
    }
    Else {
        Rear++
        queue[rear] = value;
        printf("Inserted %d\n", value);
    }
}

// Dequeue operation
void dequeue() {
    if (front == -1 || front > rear) {
        printf("Queue Underflow!\n");
    } else if front == rear
        Front = rear = -1
    Else
        printf("Deleted %d\n", queue[front++]);
}

// Display the queue
void display() {
    if (front == -1 || front > rear) {
        printf("Queue is empty.\n");
    } else {
        printf("Queue elements: ");
        for (int i = front; i <= rear; i++) {
            printf("%d ", queue[i]);
        }
        printf("\n");
    }
}

// Main function
int main() {
    enqueue(10);
    enqueue(20);
    enqueue(30);
    display();

    dequeue();
    display();

    enqueue(40);
    enqueue(50);
    enqueue(60);
    display();

    return 0;
}
```

// Should show overflow

Implementation of Queue using array

```
[Start]
|
v
[Initialize Queue]
- Set front = -1, rear = -1
|
v
[Main Loop]
|
v
[Choose Operation]
|-----|-----|
|         |         |
v         v         v
[Enqueue]  [Dequeue]  [Display]
|         |         |
v         v         v
[Is Queue Full?] [Is Queue Empty?] [Is Queue Empty?]
| Yes          | Yes          | Yes
| v            | v            | v
| [Print "Queue is | [Print "Queue is | [Print "Queue is
| full"]        | empty"]      | empty"]
| v            | v            | v
| [End Operation] | [End Operation] | [End Operation]
|              |              |
| No           | No           | No
| v            | v            | v
| [Input Element] | [Remove element | [Print elements
| rear = (rear + 1) at front]   from front to
| queue[rear] = | front = (front + 1) rear]
| element       | Return element] |
| v            | v            | v
| [Increment count] | [Decrement count] | [End Display]
| v              | v              |
| [End Enqueue]   | [End Dequeue]   |
|
v
[Repeat or End]
|
v
[End]
```

Queue

- Size:** 5 elements.

- Operations:**

- 1.**Enqueue(10):**

- 2.**Enqueue(20):**

- 3.**Dequeue():**

- 4.**Enqueue(30):**

- 5.**Dequeue():**

- 6.**Enqueue(40):**

- 7.**Enqueue(50) :**

- Final State:**

Queue

- Size:** 5 elements.

- Initial State:** front = 0, rear = -1, array = [_, _, _, _, _].

- Operations:**

- 1.**Enqueue(10):**

- rear = 0, array = [10, _, _, _, _].

- Output: "Enqueued 10".

- 2.**Enqueue(20):**

- rear = 1, array = [10, 20, _, _, _].

- Output: "Enqueued 20".

- 3.**Dequeue():**

- front = 1, data = 10, array = [_, 20, _, _, _].

- Output: "Dequeued 10".

- 4.**Enqueue(30):**

- rear = 2, array = [_, 20, 30, _, _].

- Output: "Enqueued 30".

- 5.**Dequeue():**

- front = 2, data = 20, array = [_, _, 30, _, _].

- Output: "Dequeued 20".

- 6.**Enqueue(40):**

- rear = 3, array = [_, _, 30, 40, _].

- Output: "Enqueued 40".

- 7.**Enqueue(50)** (Attempt):

- rear = 4, but next would be 5 (out of bounds), so "Queue is full".

- Final State:** front = 2, rear = 3, array = [_, _, 30, 40, _].

- Notes:** Space is wasted after dequeuing; no reuse of the first two positions.

Queue

- Enqueue(2):
- Enqueue(4):
- Enqueue(6):
- Enqueue(8):
- Enqueue(10):
- Dequeue():
- Dequeue():
- Enqueue(12)

Queue

Example 2: Filling and Emptying the Queue

• **Initial State:** front = 0, rear = -1, array = [_, _, _, _].

• **Operations:**

- **Enqueue(2):**
 - rear = 0, array = [2, _, _, _].
 - Output: "Enqueued 2".
- **Enqueue(4):**
 - rear = 1, array = [2, 4, _, _].
 - Output: "Enqueued 4".
- **Enqueue(6):**
 - rear = 2, array = [2, 4, 6, _, _].
 - Output: "Enqueued 6".
- **Enqueue(8):**
 - rear = 3, array = [2, 4, 6, 8, _].
 - Output: "Enqueued 8".

- **Enqueue(10):**
 - rear = 4, array = [2, 4, 6, 8, 10].
 - Output: "Enqueued 10".
- **Dequeue():**
 - front = 1, data = 2, array = [_, 4, 6, 8, 10].
 - Output: "Dequeued 2".
- **Dequeue():**
 - front = 2, data = 4, array = [_, _, 6, 8, 10].
 - Output: "Dequeued 4".
- **Enqueue(12) (Attempt):**
 - rear = 4, but next would be 5 (out of bounds), so "Queue is full".

• **Final State:** front = 2, rear = 4, array = [_, _, 6, 8, 10].

Queue

- **Enqueue(7):**
- **Enqueue(14):**
- **Enqueue(21):**
- **Dequeue():**
- **Dequeue():**
- **Enqueue(28):**
- **Enqueue(35):**

Queue

Example 3: Partial Dequeue and Re-Enqueue

• **Initial State:** front = 0, rear = -1, array = [_, _, _, _].

• **Operations:**

- **Enqueue(7):**
 - rear = 0, array = [7, _, _, _].
 - Output: "Enqueued 7".
- **Enqueue(14):**
 - rear = 1, array = [7, 14, _, _].
 - Output: "Enqueued 14".
- **Enqueue(21):**
 - rear = 2, array = [7, 14, 21, _].
 - Output: "Enqueued 21".
- **Dequeue():**
 - front = 1, data = 7, array = [_, 14, 21, _].
 - Output: "Dequeued 7".

- **Dequeue():**
 - front = 2, data = 14, array = [_, _, 21, _].
 - Output: "Dequeued 14".
- **Enqueue(28):**
 - rear = 3, array = [_, _, 21, 28, _].
 - Output: "Enqueued 28".
- **Enqueue(35):**
 - rear = 4, array = [_, _, 21, 28, 35].
 - Output: "Enqueued 35".

• **Final State:** front = 2, rear = 4, array = [_, _, 21, 28, 35].

Queue Implementation Methods:

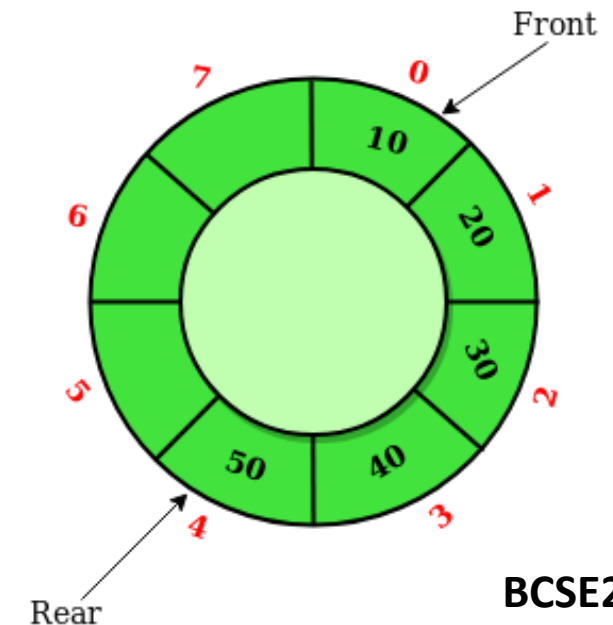
1. Using Arrays (Static Queue)

- **Fixed size:** Queue size is limited to the size of the array.
- **Operations:** enqueue, dequeue, display, isFull, isEmpty
- **Drawbacks:**
 - Cannot reuse freed space after dequeuing unless circular logic is added.
 - Wastes memory if not fully utilized.

Used for: Simple applications, fixed memory environments.

2. Using Circular Arrays (Circular Queue)

- Overcomes limitations of linear array queue by wrapping around the indices.
- rear and front are incremented in circular fashion using modulus:
 - $\text{rear} = (\text{rear} + 1) \% \text{SIZE}$
- Efficient use of array space.
- **Used for:** CPU scheduling, buffers, real-time systems.

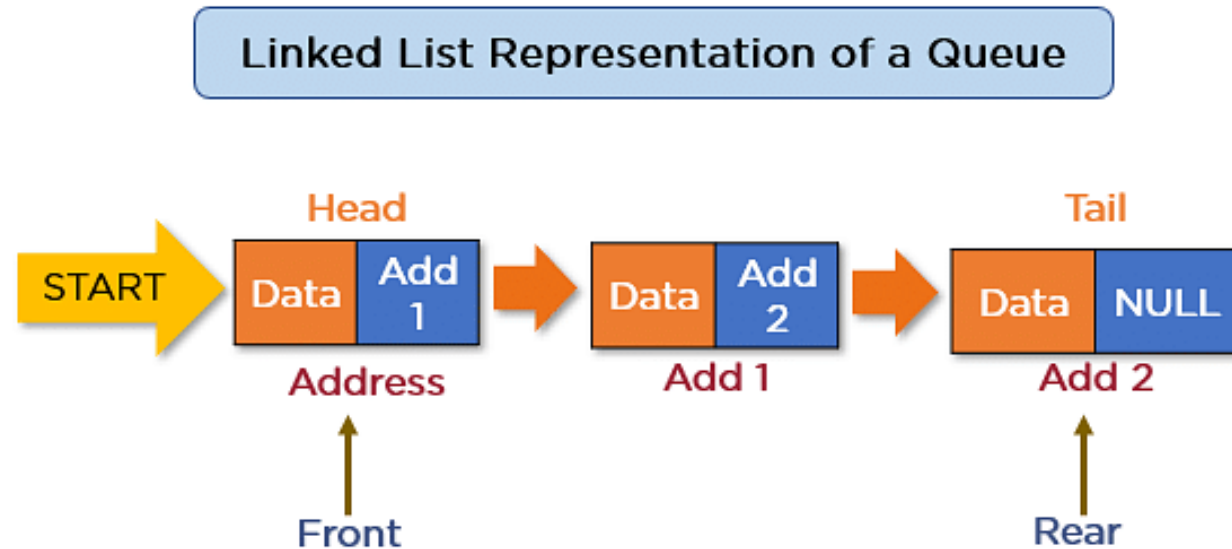


Queue

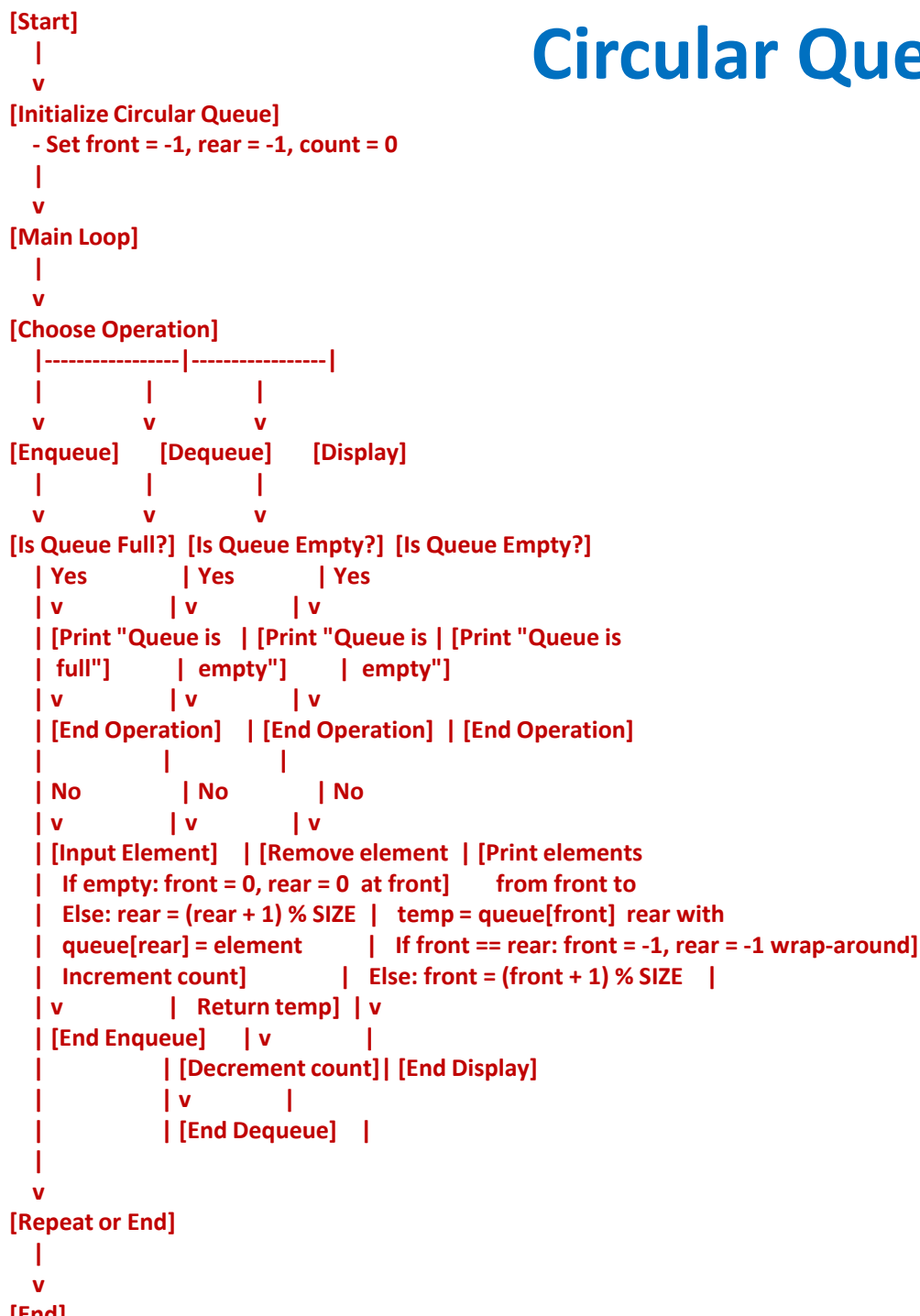
3. Using Linked List (Dynamic Queue)

- Each element (node) has data and pointer to the next node.
- No fixed size: grows dynamically.
- No memory wastage.

Used for: Flexible memory management, when number of elements is unknown or dynamic.



Circular Queue



Circular Queue

```
#include <stdio.h>
```

```
#define SIZE 5
```

```
int queue[SIZE];
```

```
int front = -1, rear = -1;
```

```
// Check if queue is full
```

```
int isFull() {  
    return ((rear + 1) % SIZE == front);  
}
```

```
// Check if queue is empty
```

```
int isEmpty() {  
    return (front == -1);  
}
```

```
// Enqueue (insert)
```

```
void enqueue(int value) {  
    if (isFull()) {  
        printf("Queue is full!\n");  
        return;  
    }  
  
    if (isEmpty()) {  
        front = rear = 0;  
    } else {  
        rear = (rear + 1) % SIZE;  
    }  
  
    queue[rear] = value;  
    printf("Inserted %d\n", value);  
}
```

```
// Dequeue (remove)
```

```
void dequeue() {  
    if (isEmpty()) {  
        printf("Queue is empty!\n");  
        return;  
    }  
  
    printf("Deleted %d\n", queue[front]);  
  
    if (front == rear) {  
        // Only one element was there  
        front = rear = -1;  
    } else {  
        front = (front + 1) % SIZE;  
    }  
}
```

Circular Queue

```
// Display
void display() {
    if (isEmpty()) {
        printf("Queue is empty!\n");
        return;
    }

    printf("Queue: ");
    int i = front;
    while (1) {
        printf("%d ", queue[i]);
        if (i == rear)
            break;
        i = (i + 1) % SIZE;
    }
    printf("\n");
}
```

```
// Main function
int main() {
    enqueue(10);
    enqueue(20);
    enqueue(30);
    enqueue(40);
    enqueue(50);
    display();

    dequeue();
    dequeue();
    display();

    enqueue(60);
    enqueue(70);
    display();

    return 0;
}
```

- enqueue(2);
- enqueue(-1);
- enqueue(5);
- enqueue(6);
- enqueue(7);
- display();
- dequeue();
- dequeue();
- enqueue(0);
- enqueue(10);
- enqueue(11);

Circular Queue

Assume a circular queue of size 5, initially:

Queue: [_, _, _, _, _]

front = -1, rear = -1

enqueue(2)

Queue: [2, _, _, _, _]

front = 0, rear = 0

enqueue(-1)

Queue: [2, -1, _, _, _]

front = 0, rear = 1

enqueue(5)

Queue: [2, -1, 5, _, _]

front = 0, rear = 2

enqueue(6)

Queue: [2, -1, 5, 6, _]

front = 0, rear = 3

enqueue(7)

Queue: [2, -1, 5, 6, 7]

front = 0, rear = 4

display()

Output: 2 -1 5 6 7

(front = 0 to rear = 4)

dequeue() → removes 2

Queue: [_, -1, 5, 6, 7]

front = 1, rear = 4

dequeue() → removes -1

Queue: [_, _, 5, 6, 7]

front = 2, rear = 4

enqueue(0) (rear wraps to 0)

Queue: [0, _, 5, 6, 7]

front = 2, rear = 0

enqueue(10)

Queue: [0, 10, 5, 6, 7]

front = 2, rear = 1

enqueue(11)

Cannot insert! Queue is full, because:

$(\text{rear} + 1) \% \text{SIZE} == \text{front}$

→ $(1 + 1) \% 5 == 2 \rightarrow \text{TRUE}$

Final State:

Queue: [0, 10, 5, 6, 7]

front = 2, rear = 1

Circular Queue

enqueue(10), enqueue(20),
enqueue(30), enqueue(40),
enqueue(50), enqueue(60),
dequeue(), dequeue(),
enqueue(60), enqueue(70)

Initial State:

Queue: [_, _, _, _, _]

front = -1, rear = -1

After enqueue(10):

Queue: [10, _, _, _, _]

front = 0, rear = 0

After enqueue(20):

Queue: [10, 20, _, _, _]

front = 0, rear = 1

After enqueue(30):

Queue: [10, 20, 30, _, _]

front = 0, rear = 2

After enqueue(40):

Queue: [10, 20, 30, 40, _]

front = 0, rear = 3

After enqueue(50):

Queue: [10, 20, 30, 40, 50]

front = 0, rear = 4

After enqueue(60):

Queue is full! (Because $(\text{rear} + 1) \% \text{SIZE} == \text{front}$)

Queue remains: [10, 20, 30, 40, 50]

After dequeue() → removes 10:

Queue: [_, 20, 30, 40, 50]

front = 1, rear = 4

After dequeue() → removes 20:

Queue: [_, _, 30, 40, 50]

front = 2, rear = 4

After enqueue(60):

Queue: [60, _, 30, 40, 50]

rear = 0 (wrapped around)

front = 2

After enqueue(70):

Queue: [60, 70, 30, 40, 50]

rear = 1

front = 2

- (a) Explain in detail about asymptotic notations (Big O, Big Ω and Big Θ) by using simple function examples with the order of growth. **1**
- b) Give the asymptotic bounds for the equation $f(n) = 2n^3 - 6n + 30$ and represent in terms of θ notation with $g(n)$ as n^3 .

a) Function Sort(array, n)

// n is the number of elements in array

for (i from 0 to n-1)

```
{  
    for (j from 0 to n-2-i)  
    {  
        if array[j] > array[j+1]  
        {  
            // Swapping array[j] and array[j+1]  
            temp = array[j]  
            array[j] = array[j+1]  
            array[j+1] = temp  
        }  
    }  
}
```

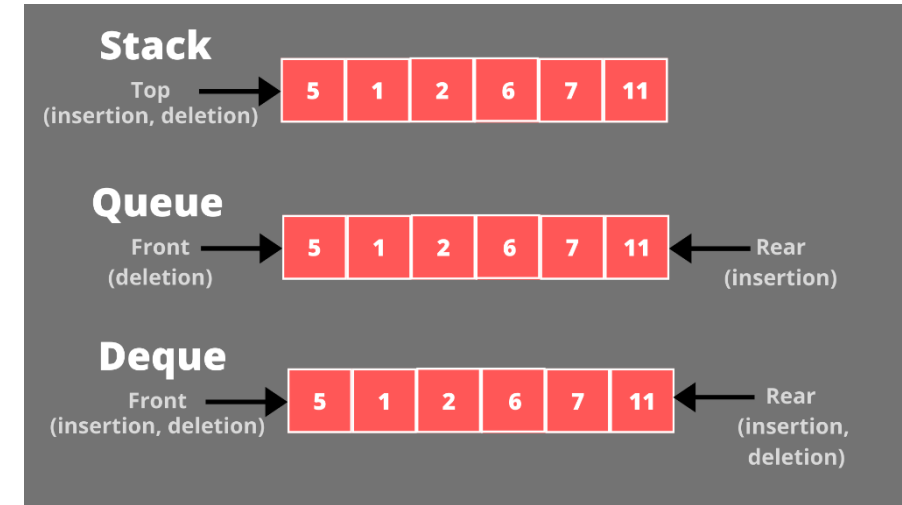
2

Find out the worst-case time complexity for this iterative algorithm using summation formulas. Assume that the input array is unsorted.

Double Ended Queue (deQueue)

- A Double-Ended Queue (Deque), often referred to as a "deque" , is a versatile data structure that allows elements to be added or removed from both ends—front and rear.
- It combines the features of a queue (FIFO: First In, First Out) and a stack (LIFO: Last In, First Out), providing flexibility for various applications.

Operation	Description
insertFront(x)	Insert element x at the front
insertRear(x)	Insert element x at the rear
deleteFront()	Delete element from the front
deleteRear()	Delete element from the rear
getFront()	Get the front element without deleting
getRear()	Get the rear element without deleting
isEmpty()	Check if deque is empty
isFull()	(If fixed-size) Check if deque is full



Queue

Types of Deques

1.Input Restricted Deque

➤ Insertion is allowed only at **one end**, but deletion is allowed from **both ends**.

2.Output Restricted Deque

➤ Deletion is allowed only at **one end**, but insertion is allowed at **both ends**.

Why Use a Deque?

Deque is a generalized form of a queue.

Efficient for scenarios requiring operations at both ends:

- Browser history
- Palindrome checking
- Sliding window problems in arrays
- Undo/Redo operations

It is used in job scheduling algorithms. (A steal algorithm in parallel computing)

Queue

```
#include <stdio.h>
#define SIZE 5

int deque[SIZE];
int front = -1, rear = -1;

// Check if deque is full
int isFull() {
    return ((front == 0 && rear == SIZE - 1) || (front == rear + 1));
}

// Check if deque is empty
int isEmpty() {
    return (front == -1);
}
```

Before insertion: [_ _ _ _ _] front = -1, rear = -1

After insertFront(10): [10,_ _ _ _] front = 0, rear = 0

After insertRear(20): [10,20,_ _ _] front = 0, rear = 1

```
// Insert element at the front
void insertFront(int value) {
    if (isFull()) {
        printf("Deque is full!\n");
        return;
    }

    if (isEmpty()) {
        front = rear = 0;
    } else if (front == 0) {
        front = SIZE - 1;
    } else {
        front--;
    }

    deque[front] = value;
}
```


Queue

```
// Insert element at the rear
void insertRear(int value) {
    if (isFull()) {
        printf("Deque is full!\n");
        return;
    }

    if (isEmpty()) {
        front = rear = 0;
    } else if (rear == SIZE - 1) {
        rear = 0;
    } else {
        rear++;
    }

    deque[rear] = value;
}
```

```
// Delete element from the front
void deleteFront() {
    if (isEmpty()) {
        printf("Deque is empty!\n");
        return;
    }

    printf("Deleted from front: %d\n", deque[front]);

    if (front == rear) {
        front = rear = -1;
    } else if (front == SIZE - 1) {
        front = 0;
    } else {
        front++;
    }
}

// Delete element from the rear
void deleteRear() {
    if (isEmpty()) {
        printf("Deque is empty!\n");
        return;
    }
}
```

```
printf("Deleted from rear: %d\n", deque[rear]);
    if (front == rear) {
        front = rear = -1;
    } else if (rear == 0) {
        rear = SIZE - 1;
    } else {
        rear--;
    }
}

// Display the deque
void display() {
    if (isEmpty()) {
        printf("Deque is empty!\n");
        return;
    }
    printf("Deque elements: ");
    int i = front;
    while (1) {
        printf("%d ", deque[i]);
        if (i == rear)
            break;
        i = (i + 1) % SIZE;
    }
    printf("\n");
}
```

1. What is the initial condition of the deque in the given implementation?

front = 0 and rear = 0

front = -1 and rear = -1

front = 1 and rear = SIZE - 1

front = SIZE - 1 and rear = 0

2. When is the deque considered FULL in the given implementation?

When front == rear

When rear == SIZE - 1

When (front == 0 && rear == SIZE - 1) || (front == rear + 1)

When front == 0

3. How does the isEmpty() function determine if the deque is empty?

Checks if front == rear

Checks if front == -1

Checks if rear == -1

Checks if front == SIZE - 1

4. In a circular deque implementation, what does the condition $\text{front} == \text{rear} + 1$ signify?

The deque has only one element.

The deque is full in a circular manner.

The deque is empty.

The front has overtaken the rear.

5. Which of the following code statements correctly inserts an element at the front in a deque implemented using a circular array?

- A) $\text{front} = (\text{front} + 1) \% \text{size}$
- B) $\text{front} = (\text{front} - 1 + \text{size}) \% \text{size}$
- C) $\text{rear} = (\text{rear} + 1) \% \text{size}$
- D) $\text{rear} = (\text{rear} - 1 + \text{size}) \% \text{size}$

6. When implementing a deque using array, how do you delete an element from the rear?

- A) $\text{rear} = (\text{rear} - 1 + \text{size}) \% \text{size}$
- B) $\text{rear} = (\text{rear} + 1) \% \text{size}$
- C) $\text{front} = \text{front} + 1$
- D) $\text{rear} = 0$

7. Consider a deque implemented using a circular array of size 6 (indexed 0 to 5).

The current state is: front = 5, rear = 2

Elements are filled from index 5, 0, 1, 2

Which condition will correctly detect that the deque is full?

- A) $(\text{front} == (\text{rear} + 1) \% \text{size})$
- B) $(\text{rear} == (\text{front} + 1) \% \text{size})$
- C) $(\text{rear} + 1) \% \text{size} == \text{front}$
- D) $(\text{front} + 1) \% \text{size} == \text{rear}$

8. In an input-restricted deque (insertion only at rear), which sequence of operations will lead to an underflow?

Given:

- Deque size = 3
- Initial state: empty

Operations:

1. Insert(10)
2. Insert(20)
3. Delete_front()
4. Delete_front()
5. Delete_rear()

- A) Only step 5 causes underflow
- B) Step 4 and 5 cause underflow
- C) Only step 4 causes underflow
- D) No underflow occurs

Queue

```
int main() {  
    insertRear(10);  
    insertRear(20);  
    insertFront(5);  
    insertFront(2);  
    display(); // Expected: 2 5 10 20  
  
    deleteRear(); // Deletes 20  
    deleteFront(); // Deletes 2  
    display(); // Expected: 5 10  
  
    return 0;  
}
```

1.

enqueueFront(2);

enqueueFront(5);

enqueueRear(-1);

enqueueRear(0);

enqueueFront(7);

enqueueFront(4);

display();

•enqueueRear(1)

•enqueueRear(2)

•enqueueFront(3)

•enqueueFront(4)

•enqueueRear(5)

•enqueueRear(6)

•display()

Queue

[_, _, _, _, _] front = -1, rear = -1

After enqueueFront(2):

[_, _, _, _, 2] front = 4, rear = 4

After enqueueFront(5):

[_, _, _, 5, 2] front = 3, rear = 4

After enqueueRear(-1):

[-1, _, _, 5, 2] front = 3, rear = 0

After enqueueRear(0):

[-1, 0, _, 5, 2] front = 3, rear = 1

After enqueueFront(7):

[-1, 0, 7, 5, 2] front = 2, rear = 1

After enqueueFront(4):

Cannot insert – Deque is full

[-1, 0, 7, 5, 2] front = 2, rear = 1

display():

[7, 5, 2, -1, 0]

[_, _, _, _, _] front = -1, rear = -1

After enqueueRear(1):

[1, _, _, _, _] front = 0, rear = 0

After enqueueRear(2):

[1, 2, _, _, _] front = 0, rear = 1

After enqueueFront(3):

[1, 2, _, _, 3] front = 4, rear = 1

After enqueueFront(4):

[1, 2, _, 4, 3] front = 3, rear = 1

After enqueueRear(5):

[1, 2, 5, 4, 3] front = 3, rear = 2

After enqueueRear(6):

Cannot insert – Deque is full

[1, 2, 5, 4, 3] front = 3, rear = 2

display():

[4, 3, 1, 2, 5]